

DEPARTMENT OF INFORMATION TECHNOLOGY

24ITT32 - PROGRAMMING IN JAVA

INDIVIDUAL PROJECT ON ACADEMIC MANAGEMENT SYSTEM

P R O J E C T R E P O R T

Submitted by

VINITHA N

25ITR174

JAVAASSIGNMENT

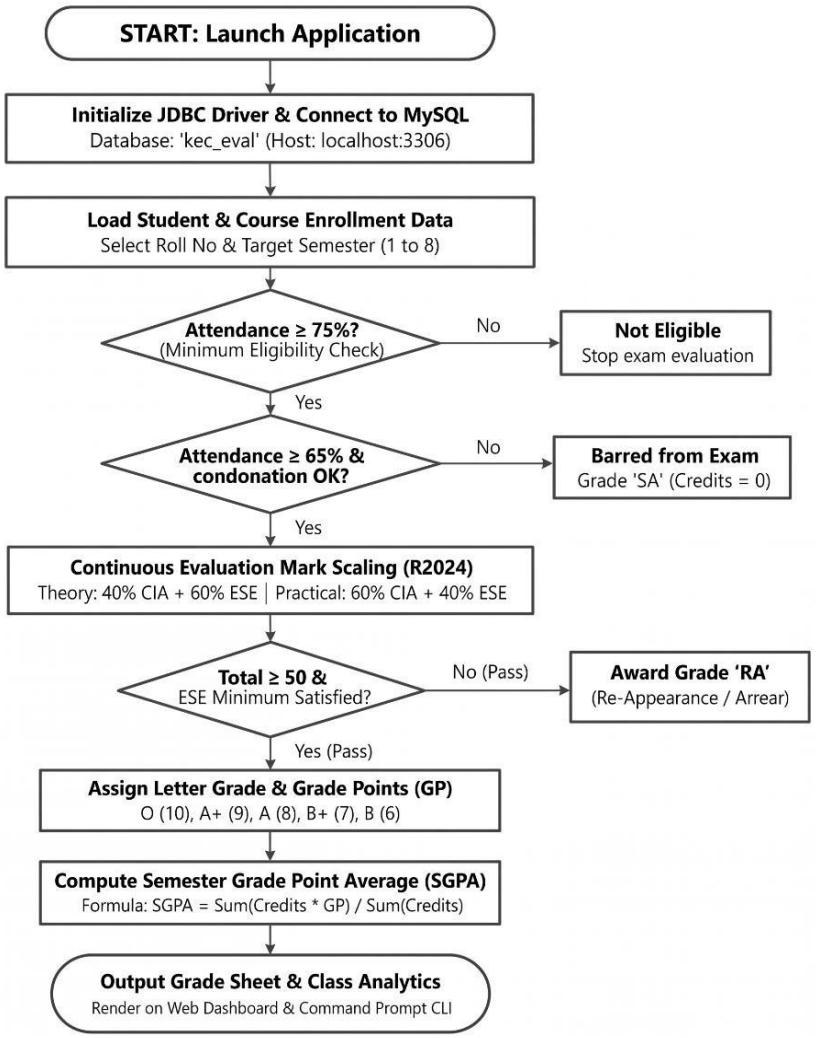
AIM:

Development of Kongu Engineering College (R2024) Student Academic & Continuous Evaluation System using Java JDBC.

OBJECTIVE:

Design and develop a database-driven Java application using JDBC to manage student records, perform incremental marks entry (Continuous Internal Assessments and End Semester Examinations), enforce attendance eligibility rules, and generate formal academic performance reports according to Kongu Engineering College (Autonomous) - Regulation 2024. System Requirements & Specifications.

FLOW CHART



PROCEDURE:

- Load the MySQL Connector/J driver to enable communication between the Java application and the MySQL database.
- Establish a connection to the kec_eval database running on the local MySQL server using DriverManager.getConnection().
- Store and retrieve student details, including roll number, name, department, and semester, along with course information such as course code, title, credits, and course type.
- Enroll students in semester courses, record the number of classes held and attended, and calculate the attendance percentage.
- Verify examination eligibility based on attendance percentage. Students with 75% or above are eligible, those with 65%–74% require medical condonation verification, and those below 65% are marked as Shortage of Attendance (SA).
- Record CIA-1, CIA-2, and assignment marks, and calculate the scaled internal assessment marks according to the specified theory and practical course weightages.
- Record end-semester examination marks out of 100, verify the minimum passing requirements, and scale the marks according to the applicable theory and practical .
- Calculate the total marks by combining scaled CIA and ESE marks. Assign letter grades and grade points according to the specified grading scale, marking students as RA if they fail the examination or score below the required passing mark.
- Generate student grade sheets, attendance shortage reports, and department-wise pass percentage reports. Display the results through the interactive web dashboard and the command prompt console**.**

DATABASE SCHEMA:

1.STUDENTS TABLE:

```
--- [1] STUDENT & COURSE MANAGEMENT ---
1. Register New Student
2. Add New Course to Catalog
3. View All Registered Students
4. View Course Catalog
5. Back to Main Menu
Select an option (1-5): 1

--- Student Registration ---
Enter Roll Number (e.g., 24ITR010): 25ITR174
Enter Student Full Name: VINITHA N
Enter Department (e.g., IT, CSE, ECE, MECH, AIDS): IT
Enter Current Semester (1-8): 3
[+] Student registered successfully: 25ITR174   | VINITHA N               | IT     | Sem: 3

------------------------------------------------------
```

2. COURSES TABLE:

```
--- [2] ENROLLMENT & ATTENDANCE TRACKING ---
1. Enroll Student into Course
2. Update Continuous Attendance (Held vs Attended)
3. Manage Medical Condonation (65% - 74.9%)
4. View All Enrollments
5. Back to Main Menu
Select an option (1-5): 1

--- Enroll Student in Course ---
Enter Roll Number: 25ITR174
Enter Course Code: 24ITC31
[+] Student 25ITR174 successfully enrolled in 24ITC31 (Enrollment ID: 17)
```

3. ENROLLMENTS TABLE:

```
--- [2] ENROLLMENT & ATTENDANCE TRACKING ---
1. Enroll Student into Course
2. Update Continuous Attendance (Held vs Attended)
3. Manage Medical Condonation (65% - 74.9%)
4. View All Enrollments
5. Back to Main Menu
Select an option (1-5): 1

--- Enroll Student in Course ---
Enter Roll Number: 25ITR174
Enter Course Code: 24ITC31
[+] Student 25ITR174 successfully enrolled in 24ITC31 (Enrollment ID: 17)
```

4. ASSESSMENT MARKS TABLE:

```
================================================================================================================
==================
                  KONGU ENGINEERING COLLEGE (AUTONOMOUS), PERUNDURAI, ERODE - 638060
                           OFFICE OF THE CONTROLLER OF EXAMINATIONS
                       ACADEMIC GRADE SHEET - REGULATION 2024 (R2024)
================================================================================================================
==================
  Roll Number : 24ITR001          Student Name : Arun Kumar S                    Semester   : 1
  Department  : IT                Degree       : B.Tech / B.E.                  Academic Yr: 2024-20
25
----------------------------------------------------------------------------------------------------------------
------------------
| Code     | Course Title                | Type | Attd %   | CIA1  | CIA2  | Asgn  | CIA(Sc) | ESE(R
) | ESE(Sc) | Total | Grd | GP | Status     |
|----------|-----------------------------|------|----------|-------|-------|-------|---------|------
--|---------|-------|-----|----|------------|
| 24CSC11  | Problem Solving and Pytho... | EMB  |   91.7%  | 88.0  | 92.0  | 90.0  | 45.0    | 90.0
  | 45.0    | 90    | A+  | 9  | PASS       |
| 24EEL11  | Basic Electrical and Elec... | PRC  |   93.3%  | 95.0  | 96.0  | 98.0  | 57.8    | 92.0
  | 36.8    | 95    | O   | 10 | PASS       |
| 24MAC11  | Matrices and Calculus        | THY  |   93.3%  | 92.0  | 95.0  | 90.0  | 37.0    | 94.0
  | 56.4    | 93    | O   | 10 | PASS       |
| 24PHC11  | Engineering Physics          | THY  |   93.3%  | 85.0  | 88.0  | 85.0  | 34.4    | 86.0
  | 51.6    | 86    | A+  | 9  | PASS       |
================================================================================================================
==================
  TOTAL CREDITS REGISTERED : 13              TOTAL CREDITS EARNED : 13
  SEMESTER GRADE POINT AVERAGE (SGPA) :  9.46 / 10.00
  EQUIVALENT PERCENTAGE               : 94.60 %
  OVERALL SEMESTER STATUS             : ALL CLEAR / PASS
================================================================================================================
==================
  Weightage Rules: THEORY: CIA 40% (Min ESE 27/60) | PRACTICAL: CIA 60% (Min ESE 20/40) | EMBEDDED: C
IA 50% (Min ESE 22.5/50)
  Grades: O (91-100), A+ (81-90), A (71-80), B+ (61-70), B (50-60), RA (Re-Appearance <50), SA (Short
age), AB (Absent)
================================================================================================================
```

SOURCE CODE:

MAINAPP.JAVA:

```
import java.io.File;

import java.sql.SQLException;

import java.util.List;

import java.util.Scanner;

/** KEC (Autonomous) Regulation 2024 - compact menu-driven Student Academic &

Continuous Evaluation System. */

public class MainApp {

    private static final StudentDAO studentDAO = new StudentDAO();

    private static final AcademicDAO academicDAO = new AcademicDAO()
```

```
    private static final Scanner scanner = new Scanner(System.in);
    private static boolean running = true;
    private static String ask(String p) { System.out.print(p); return scanner.nextLine().trim(); }
    private static int askInt(String p) { return Integer.parseInt(ask(p)); }
    private static double askDbl(String p) { return Double.parseDouble(ask(p)); }
    private static boolean yes(String p) { return ask(p).equalsIgnoreCase("Y"); }
    private static void rule(int n) { System.out.println("-".repeat(n)); }
    private interface Task { void go() throws Exception; }
    private static void safe(String ctx, Task t) { try { t.go(); } catch (Exception e) { fail(ctx, e);
} }
    private static void fail(String ctx, Exception e) { System.err.println("[!] Error " + ctx + ": "
+ e.getMessage()); }
    private static void menu(String title, String[] items, Runnable... acts) {
        System.out.println("\n--- " + title + " ---");
        for (String i : items) System.out.println(i);
        try { int k = askInt("Select an option (1-" + items.length + "): ") - 1; if (k >= 0 && k <
acts.length) acts[k].run(); else if (k != items.length - 1) System.out.println("[!] Invalid
choice."); } catch (NumberFormatException e) { System.out.println("[!] Invalid choice."); }
    }
    private static Enrollment findEnrollment() throws Exception {
        String roll = ask("Enter Student Roll Number: ").toUpperCase(), code = ask("Enter
Course Code: ").toUpperCase();
        Enrollment e = studentDAO.getEnrollment(roll, code);
        if (e == null) System.out.println("[!] No enrollment found for " + roll + " in " + code);
        return e;
    }
    public static void main(String[] args) {
```

```
 System.out.println("=".repeat(80) + "\n   KONGU ENGINEERING COLLEGE (AUTONOMOUS) - ERODE 638060\n   STUDENT ACADEMIC & CONTINUOUS EVALUATION SYSTEM (REGULATION 2024)\n" + "=".repeat(80));
        try { DBConnection.initializeDatabaseIfRequired(); System.out.println("[+] Database Connection Established (kec_eval_db)"); } catch (Exception e) { System.err.println("[!] DB Init warning: " + e.getMessage()); }
        if (args.length > 0 && args[0].equalsIgnoreCase("--demo")) { runAutomatedDemonstration(); return; }
        String[] items = {"1. Student & Course Management", "2. Enrollment & Attendance Tracking", "3. Incremental Marks Entry (Phase 1, 2, 3)", "4. Student Semester Mark Sheet (SGPA & %)",
                "5. Attendance Defaulter Report (SA Flagged)", "6. Class Performance & Result Summary", "7. Run End-to-End Demonstration", "8. Exit System"};
        while (running) menu("MAIN MENU", items, MainApp::studentCourseMenu, MainApp::attendanceMenu, MainApp::marksMenu, MainApp::markSheet, MainApp::defaulters,
                MainApp::classSummary, MainApp::runAutomatedDemonstration, () -> { System.out.println("\n[+] Exiting KEC Academic Evaluation System. Goodbye!"); running = false; }); }
    private static void studentCourseMenu() {
        menu("STUDENT & COURSE MANAGEMENT", new String[]{"1. Register New Student", "2. Add New Course", "3. View All Students", "4. View Course Catalog", "5. Back"},
                MainApp::registerStudent, MainApp::addCourse, MainApp::viewAllStudents, MainApp::viewCourseCatalog); }
    private static void registerStudent() { safe("registering student", () -> {
            String roll = ask("Enter Roll Number (e.g., 24ITR010): ");
```

```
 if (roll.isEmpty() || studentDAO.getStudentByRollNo(roll) != null) { System.out.println("[!] Roll number is empty or already exists!"); return; }
        Student s = new Student(roll, ask("Enter Full Name: "), ask("Enter Department (IT, CSE, ECE...): ").toUpperCase(), askInt("Enter Current Semester (1-8): "));
        System.out.println(studentDAO.addStudent(s) ? "[+] Student registered: " + s : "[!] Failed to register student."); });}
    private static void addCourse() { safe("adding course", () -> {
        String code = ask("Enter Course Code (e.g., 24ITC31): ").toUpperCase();
        if (academicDAO.getCourse(code) != null) { System.out.println("[!] Course code already exists!"); return; }
        String title = ask("Enter Course Title: "); int credits = askInt("Enter Credits (1-6): ");
        String t = ask("Enter Course Type (1: THEORY, 2: PRACTICAL, 3: EMBEDDED): ");
        CourseType type = t.equals("2") || t.equalsIgnoreCase("PRACTICAL") ? CourseType.PRACTICAL : t.equals("3") || t.equalsIgnoreCase("EMBEDDED") ? CourseType.EMBEDDED : CourseType.THEORY;
        Course c = new Course(code, title, credits, type, askInt("Enter Semester (1-8): "));
        if (!academicDAO.addCourse(c)) { System.out.println("[!] Failed to add course."); return; }
        System.out.printf("[+] Course added: %s\n    Rules: CIA %.0f%% | ESE %.0f%% | ESE Min Pass: %.1f\n", c, type.getCiaWeightage() * 100, type.getEseWeightage() * 100, type.getEseMinScaledScore()); });}
    private static void viewAllStudents() { safe("retrieving students", () -> {
        List<Student> l = studentDAO.getAllStudents(); rule(60); System.out.printf("%-10s | %-25s | %-6s | %-8s\n", "Roll No", "Student Name", "Dept", "Semester"); rule(60);
        for (Student s : l) System.out.printf("%-10s | %-25s | %-6s | Sem %d\n", s.getRollNo(), s.getName(), s.getDepartment(), s.getSemester());
```

```
rule(60); System.out.println("Total Students Registered: " + l.size()); });}
    private static void viewCourseCatalog() { safe("retrieving courses", () -> {
        List<Course> l = academicDAO.getAllCourses(); rule(85); System.out.printf("%-10s |
%-38s | %-4s | %-9s | %-8s\n", "Code", "Course Title", "Cred", "Type", "Semester"); rule(85);
        for (Course c : l) System.out.printf("%-10s | %-38s | %-4d | %-9s | Sem %d\n",
c.getCourseCode(), c.getTitle(), c.getCredits(), c.getCourseType(), c.getSemester());
        rule(85); System.out.println("Total Courses in Catalog: " + l.size()); });}
    private static void attendanceMenu() {
       menu("ENROLLMENT & ATTENDANCE TRACKING", new String[]{"1. Enroll
Student into Course", "2. Update Attendance (Held vs Attended)", "3. Manage Medical
Condonation (65-74.9%)", "4. View All Enrollments", "5. Back"},
            MainApp::enrollStudent, MainApp::updateAttendance,
MainApp::manageCondonation, MainApp::viewAllEnrollments); }
    private static void enrollStudent() { safe("enrolling", () -> {
        String roll = ask("Enter Roll Number: ").toUpperCase(), code = ask("Enter Course
Code: ").toUpperCase();
        Student s = studentDAO.getStudentByRollNo(roll);
        if (s == null || academicDAO.getCourse(code) == null) { System.out.println("[!]
Student or course not found."); return; }
        try { System.out.println("[+] Enrolled " + roll + " in " + code + " (Enrollment ID: " +
studentDAO.enrollStudent(roll, code, s.getSemester()) + ")"); }
        catch (SQLException e) { System.out.println(e.getMessage().contains("Duplicate") ?
"[!] Already enrolled in this course!" : "[!] Enrollment error: " + e.getMessage()); } });}
    private static void updateAttendance() { safe("updating attendance", () -> {
        Enrollment en = findEnrollment(); if (en == null) return;
        System.out.printf("Current: Held %d, Attended %d (%.1f%%)\n",
en.getClassesHeld(), en.getClassesAttended(), en.getAttendancePercentage());
```

```
 int held = askInt("Enter Total Classes Held: "), att = askInt("Enter Total Classes Attended: ");
        double pct = held > 0 ? att * 100.0 / held : 0.0; boolean cond =
en.isMedicalCondonation(); System.out.printf("New Attendance Percentage: %.1f%%\n",
pct);
        if (pct >= 65.0 && pct < 75.0) cond = yes("In conditional range (65-74.9%). Medical
condonation approved? (Y/N): ");
        if (studentDAO.updateAttendance(en.getEnrollmentId(), held, att, cond)) {
academicDAO.recalculateAndStoreEvaluation(en.getEnrollmentId()); System.out.println("[+]
Attendance updated and evaluation recalculated."); } });}
    private static void manageCondonation() { safe("updating condonation", () -> {
        Enrollment en = findEnrollment(); if (en == null) return;
        double pct = en.getAttendancePercentage();
        System.out.printf("%s (%s) | %s | Attendance: %.1f%% | Condonation: %s\n",
en.getStudentName(), en.getRollNo(), en.getCourseCode(), pct, en.isMedicalCondonation() ?
"APPROVED" : "NOT APPROVED");
        if (pct < 65.0) { System.out.println("[!] Regulation 2024: Attendance < 65%
CANNOT be condoned. Permanently Barred (SA) from ESE.");
 return; }
        if (pct >= 75.0) { System.out.println("[+] Attendance >= 75%. Condonation not
required.");
 return; }
        boolean ok = yes("APPROVE medical condonation? (Y/N): ");
        studentDAO.setMedicalCondonation(en.getEnrollmentId(), ok);
academicDAO.recalculateAndStoreEvaluation(en.getEnrollmentId());
        System.out.println("[+] Medical Condonation status: " + (ok ? "APPROVED" :
"REVOKED")); });}
    private static void viewAllEnrollments() { safe("retrieving enrollments", () -> {
```

```
 List<Enrollment> l = studentDAO.getAllEnrollments(); rule(85); System.out.printf("%-4s |
%-10s | %-9s | %-6s | %-8s | %-8s | %-20s\n", "ID", "Roll No",

"Course", "Held", "Attended", "Attd %", "Eligibility Status"); rule(85);
        for (Enrollment e : l) System.out.printf("%-4d | %-10s | %-9s | %-6d | %-8d |
%6.1f%%  | %-20s\n", e.getEnrollmentId(), e.getRollNo(), e.getCourseCode(),
e.getClassesHeld(), e.getClassesAttended(), e.getAttendancePercentage(),
e.getEligibilityStatus());
        rule(85); System.out.println("Total Course Enrollments: " + l.size()); });}
   private static void marksMenu() {
      menu("INCREMENTAL MARKS ENTRY", new String[]{"1. Phase 1 (Mid-Sem): CIA
Test 1 & 2", "2. Phase 2 (End-Sem): Assignments/Quizzes/Labs", "3. Phase 3 (Post-Sem):
ESE / Mark Absent", "4. Back"},
            () -> enterMarks(1), () -> enterMarks(2), () -> enterMarks(3)); }
   private static void enterMarks(int phase) { safe("entering marks", () -> {
        Enrollment en = findEnrollment(); if (en == null) return;
        int id = en.getEnrollmentId();
        if (phase == 1) {
            double c1 = askDbl("Enter CIA Test 1 Marks (0-100): "), c2 = askDbl("Enter CIA
Test 2 Marks (0-100): ");
            if (academicDAO.recordPhase1Marks(id, c1, c2)) System.out.printf("[+] Phase 1
recorded. Internal Average: %.2f / 100\n", (c1 + c2) / 2.0);
        } else if (phase == 2) {
            if (!academicDAO.recordPhase2Marks(id, askDbl("Enter Assignment/Quiz/Lab
Marks (0-100): "))) return;
            AssessmentMark m = academicDAO.getMarksByEnrollment(id);
            System.out.printf("[+] Phase 2 recorded. Final CIA: %.2f / 100 | Scaled for %s:
```

```
%.2f\n", EvaluationEngine.computeRawCia100(m.getCia1Marks(), m.getCia2Marks(),
m.getAssignmentMarks()), en.getCourseType(), m.getCiaScaled());
        } else if (!en.isEligibleForEse()) {
            System.out.printf("[!] %.1f%% attendance - BARRED from ESE. Grade 'SA'
awarded (KEC Regulation 2024).\n", en.getAttendancePercentage());
            academicDAO.recordPhase3Marks(id, false, null);
        } else {
            boolean absent = yes("Was the student ABSENT in ESE? (Y/N): ");
            Double ese = absent ? null : Double.valueOf(askDbl("Enter ESE Raw Marks (0-
100): "));
            if (!academicDAO.recordPhase3Marks(id, absent, ese)) return;
            AssessmentMark m = academicDAO.getMarksByEnrollment(id);
            System.out.printf("[+] ESE recorded. %s | Total: %.1f | Grade: %s | GP: %d |
Status: %s\n", en.getCourseCode(), m.getTotalMarks(), m.getGrade(), m.getGradePoint(),
m.getStatus());
        } });}
    private static void markSheet() { safe("generating mark sheet", () -> {
        String roll = ask("\nEnter Student Roll Number: ").toUpperCase(); int sem =
askInt("Enter Semester (e.g., 1): ");
        SemesterMarkSheet sheet = academicDAO.getStudentSemesterMarkSheet(roll, sem);
        if (sheet == null) { System.out.println("[!] Student not found: " + roll); return; }
        ConsoleReportFormatter.printSemesterMarkSheet(sheet);
        if (yes("Export as official HTML Report? (Y/N): ")) System.out.println("[+] Exported
to: " + HtmlReportGenerator.generateMarkSheetHtml(sheet, "reports/" + roll + "_Sem" + sem
+ "_MarkSheet.html")); });}
    private static void defaulters() { safe("generating defaulter report", () -> {
```

```
    String s = ask("\nFilter by Semester (Enter for ALL): "), d = ask("Filter by Department
(Enter for ALL): ");
        Integer sem = s.isEmpty() ? null : Integer.parseInt(s); if (d.isEmpty()) d = "ALL"
ConsoleReportFormatter.printDefaulterReport(academicDAO.getAttendanceDefaulters(sem,
d), sem, d); });}
    private static void classSummary() { safe("generating class summary", () -> {
        String d = ask("\nEnter Department (e.g., IT, CSE, or ALL): "); if (d.isEmpty()) d =
"IT";
ConsoleReportFormatter.printClassSummaryReport(academicDAO.getClassPerformanceSum
mary(askInt("Enter Semester: "), d)); });}
    public static void runAutomatedDemonstration() {
        System.out.println("\n" + "#".repeat(80) + "\n  AUTOMATED DEMONSTRATION -
KEC REGULATION 2024\n" + "#".repeat(80));
        try {
            String roll = "24ITR099", code = "24ITC32";
            if (studentDAO.getStudentByRollNo(roll) == null) studentDAO.addStudent(new
Student(roll, "Naveen Kumar M", "IT", 1));
            if (academicDAO.getCourse(code) == null) academicDAO.addCourse(new
Course(code, "Data Structures and Algorithms", 4, CourseType.EMBEDDED, 1));
            Enrollment en = studentDAO.getEnrollment(roll, code);
            int id = en == null ? studentDAO.enrollStudent(roll, code, 1) : en.getEnrollmentId();
            studentDAO.updateAttendance(id, 60, 52, false);
            System.out.println("[1-2] Student/course ready, enrolled; attendance 52/60 = 86.7%
(Eligible).\n[3] Marks: CIA1=88, CIA2=92 | Assignment/Lab=85 | ESE=82");
            academicDAO.recordPhase1Marks(id, 88.0, 92.0);
academicDAO.recordPhase2Marks(id, 85.0); academicDAO.recordPhase3Marks(id, false,
82.0);
```

```
   AssessmentMark m = academicDAO.getMarksByEnrollment(id);
            System.out.printf("  -> Scaled CIA: %.2f | Scaled ESE: %.2f | Total: %.0f | Grade: %s
| GP: %d | Status: %s\n", m.getCiaScaled(), m.getEseScaled(), m.getTotalMarks(),
m.getGrade(), m.getGradePoint(), m.getStatus());
            System.out.println("\n[4] Grade Sheet for 24ITR001 (Arun Kumar S):");
            SemesterMarkSheet sheet =
academicDAO.getStudentSemesterMarkSheet("24ITR001", 1);
ConsoleReportFormatter.printSemesterMarkSheet(sheet);
            String html = "reports/24ITR001_Sem1_MarkSheet.html";
HtmlReportGenerator.generateMarkSheetHtml(sheet, html);
            System.out.println("  -> HTML Grade Sheet: " + new File(html).getAbsolutePath() +
"\n[5] Attendance Defaulter Report (IT, Sem 1):");

ConsoleReportFormatter.printDefaulterReport(academicDAO.getAttendanceDefaulters(1,
"IT"), 1, "IT");
            System.out.println("\n[6] Class Performance Summary (IT, Sem 1):");
ConsoleReportFormatter.printClassSummaryReport(academicDAO.getClassPerformanceSum
mary(1, "IT"));
            System.out.println("\n[+] DEMONSTRATION COMPLETED SUCCESSFULLY!");
        } catch (Exception e) { System.err.println("[!] Demo error: " + e.getMessage());
e.printStackTrace(); }
    }
}
```

DB CONNECTION.JAVA:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

/**
 * Kongu Engineering College (Autonomous) - Regulation 2024
 * DBConnection: JDBC Connection Management for kec_eval_db.
 * Handles driver registration, connection pooling/lifecycle, and schema verification.
 */
public class DBConnection {

    private static final String DEFAULT_HOST = "localhost";
    private static final String DEFAULT_PORT = "3306";
    private static final String DEFAULT_DB = "kec_eval_db";
    private static final String DEFAULT_USER = "root";
    private static final String DEFAULT_PASSWORD = "";

    private static String url;
    private static String user;
    private static String password;

    static {
        // Allow runtime overrides via System properties or Environment variables
        String host = System.getProperty("DB_HOST",
System.getenv().getOrDefault("DB_HOST", DEFAULT_HOST));
```

```
        String port = System.getProperty("DB_PORT",

System.getenv().getOrDefault("DB_PORT", DEFAULT_PORT));
        String dbName = System.getProperty("DB_NAME",
System.getenv().getOrDefault("DB_NAME", DEFAULT_DB));
        user = System.getProperty("DB_USER", System.getenv().getOrDefault("DB_USER",
DEFAULT_USER));
        password = System.getProperty("DB_PASS",
System.getenv().getOrDefault("DB_PASS", DEFAULT_PASSWORD));

        url =
String.format("jdbc:mysql://%s:%s/%s?useSSL=false&allowPublicKeyRetrieval=true&server
Timezone=UTC&characterEncoding=UTF-8",
                host, port, dbName);

        try {
            // Load MySQL Connector/J driver
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            try {
                // Fallback for older drivers
                Class.forName("com.mysql.jdbc.Driver");
            } catch (ClassNotFoundException ex) {
                System.err.println("[DBConnection] WARNING: MySQL JDBC Driver not found
on classpath!");
            }
        }
```

```
}
  /**

* Obtains a new JDBC Connection.
   */
   public static Connection getConnection() throws SQLException {
      return DriverManager.getConnection(url, user, password);
   }

   /**
    * Tests database connectivity.
    */
   public static boolean testConnection() {
      try (Connection conn = getConnection()) {
         return conn != null && !conn.isClosed();
      } catch (SQLException e) {
         System.err.println("[DBConnection] Connection failed: " + e.getMessage());
         return false;
      }
   }

   /**
    * Safely closes resources.
    */
   public static void close(AutoCloseable... closeables) {
      for (AutoCloseable c : closeables) {
         if (c != null) {
```

```
try {
                c.close();
            } catch (Exception ignored) {
            }

}
    }
 }

 /**
  * Verifies that the required tables exist, or creates them if missing.
  */
 public static void initializeDatabaseIfRequired() {
    String testQuery = "SELECT 1 FROM students LIMIT 1";
    try (Connection conn = getConnection(); Statement stmt = conn.createStatement()) {
        stmt.executeQuery(testQuery);
    } catch (SQLException e) {
        System.out.println("[DBConnection] Tables not found. Initializing schema...");
        createSchemaTables();
    }
 }

 private static void createSchemaTables() {
    String createStudents = "CREATE TABLE IF NOT EXISTS students ("
            + "roll_no VARCHAR(15) PRIMARY KEY, "
            + "name VARCHAR(100) NOT NULL, "
            + "department VARCHAR(20) NOT NULL, "
```

```
+ "semester INT NOT NULL"
            + ") ENGINE=InnoDB;";

        String createCourses = "CREATE TABLE IF NOT EXISTS courses ("
            + "course_code VARCHAR(15) PRIMARY KEY, "
            + "title VARCHAR(150) NOT NULL, "
            + "credits INT NOT NULL, "
            + "course_type ENUM('THEORY', 'PRACTICAL', 'EMBEDDED') NOT NULL, "
            + "semester INT NOT NULL"
            + ") ENGINE=InnoDB;";

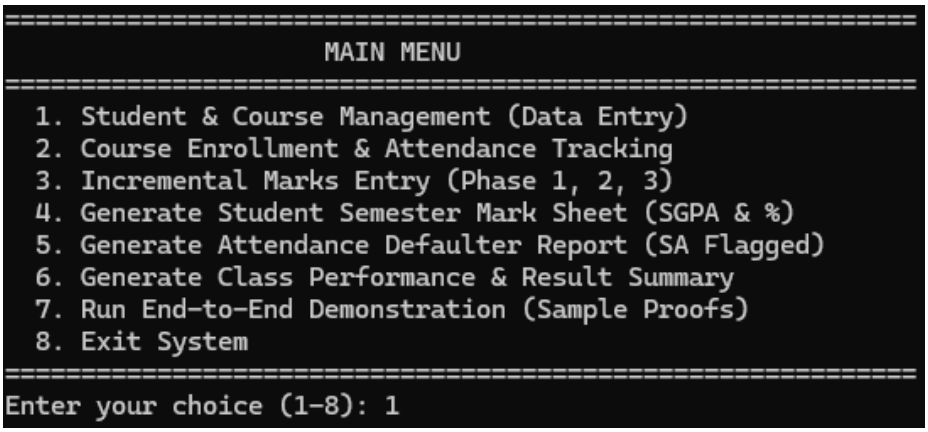
        String createEnrollments = "CREATE TABLE IF NOT EXISTS enrollments ("
            + "enrollment_id INT AUTO_INCREMENT PRIMARY KEY, "
            + "roll_no VARCHAR(15) NOT NULL, "
            + "course_code VARCHAR(15) NOT NULL, "
            + "semester INT NOT NULL, "
            + "classes_held INT DEFAULT 0, "
            + "classes_attended INT DEFAULT 0, "
            + "medical_condonation BOOLEAN DEFAULT FALSE, "
            + "UNIQUE KEY uq_student_course (roll_no, course_code), "
            + "FOREIGN KEY (roll_no) REFERENCES students(roll_no) ON DELETE
CASCADE, "
            + "FOREIGN KEY (course_code) REFERENCES courses(course_code) ON
DELETE CASCADE"
            + ") ENGINE=InnoDB;";

        String createMarks = "CREATE TABLE IF NOT EXISTS assessment_marks ("
```

```
 + "mark_id INT AUTO_INCREMENT PRIMARY KEY, "
            + "enrollment_id INT NOT NULL UNIQUE, "
            + "cia1_marks DECIMAL(5,2) NULL, "
            + "cia2_marks DECIMAL(5,2) NULL, "
            + "assignment_marks DECIMAL(5,2) NULL, "
            + "is_absent_ese BOOLEAN DEFAULT FALSE, "
            + "ese_marks DECIMAL(5,2) NULL, "
            + "cia_scaled DECIMAL(5,2) NULL, "
            + "ese_scaled DECIMAL(5,2) NULL, "
            + "total_marks DECIMAL(5,2) NULL, "
            + "grade VARCHAR(5) NULL, "
            + "grade_point INT NULL, "
            + "status VARCHAR(20) NULL, "
            + "FOREIGN KEY (enrollment_id) REFERENCES enrollments(enrollment_id)
ON DELETE CASCADE"
            + ") ENGINE=InnoDB;";
  try (Connection conn = getConnection(); Statement stmt = conn.createStatement()) {
          stmt.executeUpdate(createStudents);
          stmt.executeUpdate(createCourses);
          stmt.executeUpdate(createEnrollments);
          stmt.executeUpdate(createMarks);
          System.out.println("[DBConnection] Schema created successfully.");
      } catch (SQLException ex) {
          System.err.println("[DBConnection] Error creating schema tables: " +
ex.getMessage());
      }
 }
```

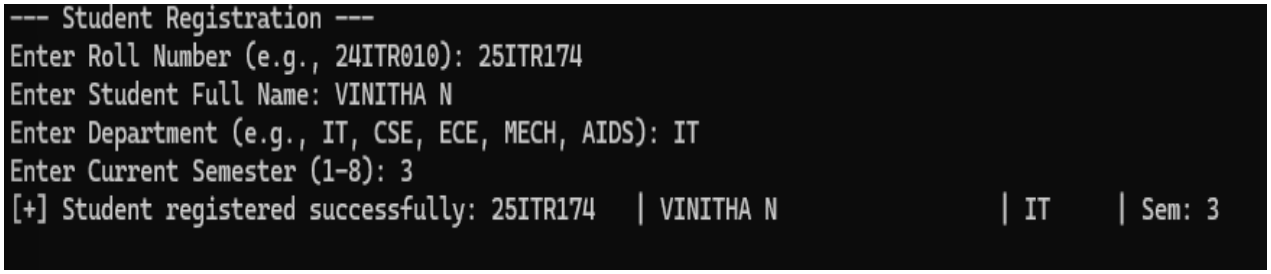
OUTPUT SCREENSHOTS:

MAIN MENU:

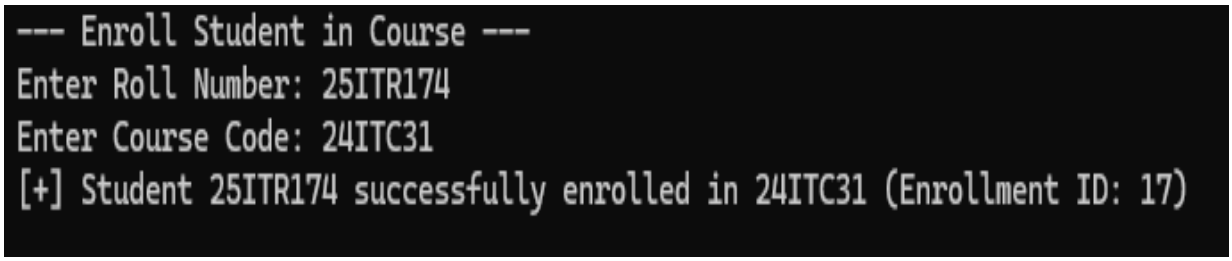


STUDENT & COURSE MANAGEMENT:

1. STUDENT REGISTRATION:



2. COURSE CATALOG:



3. COURSE ENROLLMENT & ATTENDANCE TRACKING:

```
--- [2] ENROLLMENT & ATTENDANCE TRACKING ---
1. Enroll Student into Course
2. Update Continuous Attendance (Held vs Attended)
3. Manage Medical Condonation (65% - 74.9%)
4. View All Enrollments
5. Back to Main Menu
Select an option (1-5): 1

--- Enroll Student in Course ---
Enter Roll Number: 25ITR174
Enter Course Code: 24ITC31
[+] Student 25ITR174 successfully enrolled in 24ITC31 (Enrollment ID: 17)
```

INCREMENTAL MARKS ENTRY:

PHASE 1 (MID-SEMESTER):

```
--- [3] INCREMENTAL MARKS ENTRY (THEN AND NOW) ---
1. Phase 1 (Mid-Semester)     : CIA Test 1 (0-100) & CIA Test 2 (0-100)
2. Phase 2 (End-of-Semester)  : Assignments / Quizzes / Lab Evaluations (0-100)
3. Phase 3 (Post-Semester)    : End Semester Examination (ESE 0-100) / Mark Absent
4. Back to Main Menu
Select Assessment Phase (1-4): 1

--- Phase 1: Mid-Semester CIA Tests Entry ---
Enter Student Roll Number: 25ITR174
Enter Course Code: 2ITC31
[!] Enrollment not found!
```

1.PHASE 2 (END OF SEMESTER - CIA):

```
--- [3] INCREMENTAL MARKS ENTRY (THEN AND NOW) ---
1. Phase 1 (Mid-Semester)     : CIA Test 1 (0-100) & CIA Test 2 (0-100)
2. Phase 2 (End-of-Semester)  : Assignments / Quizzes / Lab Evaluations (0-100)
3. Phase 3 (Post-Semester)    : End Semester Examination (ESE 0-100) / Mark Absent
4. Back to Main Menu
Select Assessment Phase (1-4): 1
```

1. PHASE 3 (POST SEMESTER):

```
--- [3] INCREMENTAL MARKS ENTRY (THEN AND NOW) ---
1. Phase 1 (Mid-Semester)     : CIA Test 1 (0-100) & CIA Test 2 (0-100)
2. Phase 2 (End-of-Semester)  : Assignments / Quizzes / Lab Evaluations (0-100)
3. Phase 3 (Post-Semester)    : End Semester Examination (ESE 0-100) / Mark Absent
4. Back to Main Menu
Select Assessment Phase (1-4): 1

--- Phase 1: Mid-Semester CIA Tests Entry ---
Enter Student Roll Number: 25ITR174
Enter Course Code: 2ITC31
[!] Enrollment not found!
```

ATTENDANCE & GRADING RULES:

1. ATTENDANCE FILTER:

```
==========================================================================================================
==========
                    KONGU ENGINEERING COLLEGE (AUTONOMOUS) - REGULATION 2024
                     ATTENDANCE DEFAULTER REPORT (SEMESTER: ALL | DEPT: ALL)
              Mandatory Threshold: >= 75% Eligible | 65-74.9% Needs Condonation | < 65% Barred (SA)
==========================================================================================================
==========
| Roll No    | Student Name         | Dept   | Course    | Held   | Attended | Attd %   | Grade | Ac
tion / Status          |
|------------|----------------------|--------|-----------|--------|----------|----------|-------|---
----------------------|
| 24ITR004   | Dharani V            | IT     | 24MAC11   | 60     | 40       |   66.7%  | SA    | Bo
rderline Shortage (... |
| 24ITR004   | Dharani V            | IT     | 24PHC11   | 45     | 30       |   66.7%  | SA    | Bo
rderline Shortage (... |
| 24ITR005   | Elango K             | IT     | 24MAC11   | 60     | 32       |   53.3%  | SA    | Cr
itical Shortage (53... |
| 24ITR005   | Elango K             | IT     | 24PHC11   | 45     | 24       |   53.3%  | SA    | Cr
itical Shortage (53... |
==========================================================================================================
==========
  Total Defaulter Cases Flagged with SA Grade: 4
==========================================================================================================
```

STUDENT SEMESTER MARK SHEET:

```
================================================================================================================
====================
                  KONGU ENGINEERING COLLEGE (AUTONOMOUS), PERUNDURAI, ERODE - 638060
                           OFFICE OF THE CONTROLLER OF EXAMINATIONS
                       ACADEMIC GRADE SHEET - REGULATION 2024 (R2024)
================================================================================================================
====================
  Roll Number : 24ITR001          Student Name : Arun Kumar S                    Semester   : 1
  Department  : IT                Degree       : B.Tech / B.E.                   Academic Yr: 2024-2
025
----------------------------------------------------------------------------------------------------------------
--------------------
| Code     | Course Title                 | Type | Attd %   | CIA1  | CIA2  | Asgn  | CIA(Sc) | ESE(
R) | ESE(Sc) | Total | Grd | GP | Status     |
|----------|------------------------------|------|----------|-------|-------|-------|---------|-----
---|---------|-------|-----|----|------------|
| 24CSC11  | Problem Solving and Pytho... | EMB  |   91.7%  | 88.0  | 92.0  | 90.0  | 45.0    | 90.0
   | 45.0    | 90    | A+  | 9  | PASS       |
| 24EEL11  | Basic Electrical and Elec... | PRC  |   93.3%  | 95.0  | 96.0  | 98.0  | 57.8    | 92.0
   | 36.8    | 95    | O   | 10 | PASS       |
| 24MAC11  | Matrices and Calculus        | THY  |   93.3%  | 92.0  | 95.0  | 90.0  | 37.0    | 94.0
   | 56.4    | 93    | O   | 10 | PASS       |
| 24PHC11  | Engineering Physics          | THY  |   93.3%  | 85.0  | 88.0  | 85.0  | 34.4    | 86.0
   | 51.6    | 86    | A+  | 9  | PASS       |
================================================================================================================
====================
  TOTAL CREDITS REGISTERED : 13              TOTAL CREDITS EARNED : 13
  SEMESTER GRADE POINT AVERAGE (SGPA) :  9.46 / 10.00
  EQUIVALENT PERCENTAGE               : 94.60 %
  OVERALL SEMESTER STATUS             : ALL CLEAR / PASS
================================================================================================================
```

DEPARTMENT WISE CLASS PERFORMANCE & RESULT SUMMARY:

```
================================================================================================
             KONGU ENGINEERING COLLEGE (AUTONOMOUS) - REGULATION 2024
                   CLASS PERFORMANCE & RESULT SUMMARY REPORT
                      Department: IT     | Semester: 1
================================================================================================
  Total Enrolled Students  : 6
  All-Clear Passed Count   : 3
  Re-Appearance (RA) Count : 0
  Attendance Shortage (SA) : 2
  Absent in ESE (AB) Count : 1
------------------------------------------------------------------------------------------------
  CLASS PASS PERCENTAGE    :  50.00 %
  CLASS AVERAGE SGPA       :   5.08 / 10.00 (Equivalent: 50.80 %)
------------------------------------------------------------------------------------------------
  10-Point Grade Distribution Across All Course Enrollments:
    [O] Outstanding (91-100) : 2       [B+] Good (61-70)    : 1
    [A+] Excellent  (81-90)  : 4       [B]  Average (50-60) : 3
    [A] Very Good   (71-80)  : 1       [RA] Arrear (<50)    : 0
    [SA] Shortage of Attd    : 4       [AB] Absent in ESE   : 1
================================================================================================
```

DATABASE:

STUDENTS DATABASE:

roll_no	name	department	semester	created_at
24CSR001	Gokulnath R	CSE	1	2026-10-27 11:51:58
24CSR002	Harini S	CSE	1	2026-10-27 11:51:58
24ITR001	Arun Kumar S	IT	1	2026-10-27 11:51:58
24ITR002	Bhavani Devi M	IT	1	2026-10-27 11:51:58
24ITR003	Chandru P	IT	1	2026-10-27 11:51:58
24ITR004	Dharani V	IT	1	2026-10-27 11:51:58
24ITR005	Elango K	IT	1	2026-10-27 11:51:58
24ITR099	Naveen Kumar M	IT	1	2026-10-27 12:01:11
25ITR174	VINITHA N	IT	3	2026-10-27 12:17:04

COURSES DATABASE:

course_code	title	credits	course_type	semester	created_at
24CSC11	Problem Solving and Python Programming	4	EMBEDDED	1	2026-10-27 11:51:58
24EEL11	Basic Electrical and Electronics Engineering Practice	2	PRACTICAL	1	2026-10-27 11:51:58
24ITC31	Object Oriented Programming using Java	3	THEORY	3	2026-10-27 11:51:58
24ITC32	Data Structures and Algorithms	4	EMBEDDED	1	2026-10-27 12:01:11
24MAC11	Matrices and Calculus	4	THEORY	1	2026-10-27 11:51:58
24PHC11	Engineering Physics	3	THEORY	1	2026-10-27 11:51:58

ENROLLMENTS DATABASE:

enrollment_id	roll_no	course_code	semester	classes_held	classes_attended	medical_condonation	enrolled_at
1	24ITR001	24MAC11	1	60	56	0	2026-10-27 11:51:58
2	24ITR001	24PHC11	1	45	42	0	2026-10-27 11:51:58
3	24ITR001	24CSC11	1	60	55	0	2026-10-27 11:51:58
4	24ITR001	24EEL11	1	30	28	0	2026-10-27 11:51:58
5	24ITR002	24MAC11	1	60	50	0	2026-10-27 11:51:58
6	24ITR002	24PHC11	1	45	38	0	2026-10-27 11:51:58
7	24ITR002	24CSC11	1	60	48	0	2026-10-27 11:51:58
8	24ITR002	24EEL11	1	30	25	0	2026-10-27 11:51:58
9	24ITR003	24MAC11	1	60	42	1	2026-10-27 11:51:58
10	24ITR003	24PHC11	1	45	32	1	2026-10-27 11:51:58
11	24ITR003	24CSC11	1	60	42	1	2026-10-27 11:51:58
12	24ITR004	24MAC11	1	60	40	0	2026-10-27 11:51:58
13	24ITR004	24PHC11	1	45	30	0	2026-10-27 11:51:58
14	24ITR005	24MAC11	1	60	32	0	2026-10-27 11:51:58
15	24ITR005	24PHC11	1	45	24	0	2026-10-27 11:51:58
16	24ITR099	24ITC32	1	60	52	0	2026-10-27 12:01:11
17	25ITR174	24ITC31	3	0	0	0	2026-10-27 12:17:57

RESULT:

Thus, the Student Academic and Continuous Evaluation System for Kongu Engineering College (Autonomous) Regulation 2024 was successfully designed, developed, and tested using Java (JDBC) and MySQL. The system accurately automates the calculation of Continuous Internal Assessments (CIA), End Semester Examinations (ESE), Letter Grades, Attendance Eligibility, and Semester Grade Point Average (SGPA).

REFERENCES:

1. Oracle, Java SE Documentation – JDBC Overview and Architecture.

https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/ .

2. Oracle, Java SE API – HttpServer.

https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html .

3. Oracle, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/ .

4. Oracle, MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/8.0/en/ .

5. University Grants Commission, Guidelines for Choice Based Credit System (CBCS).

https://www.ugc.gov.in/pdfnews/8023719_Guidelines-for-CBCS.pdf .

6. Chart.js, Chart.js Documentation. https://www.chartjs.org/docs/latest/ .

7. Tailwind CSS, Tailwind CSS Documentation. https://tailwindcss.com/docs .

8. GeeksforGeeks, Establishing JDBC Connection in Java.

https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/ .

9. YouTube, Java JDBC with MySQL Tutorial Videos.

https://www.youtube.com/results?search_query=Java+JDBC+MySQL+tutorial .

10. Anthropic, Claude AI Assistant (used for project ideas and guidance).

https://claude.ai/ .

GITHUB:

LINK:	**https://github.com/vinithanagarajv-boop/JAVA-ASSIGNMENT-JDBC-.git**